

ENTERPRISE-LEVEL PROJECT - WEB DEVELOPMENT (MERN STACK)

Project 1: Multi-Tenant Distributed Workflow & Saga Orchestration Engine

Problem Statement:

Design and implement a resilient, distributed workflow automation engine (similar to Temporal or Zapier) where enterprise users build complex multi-step integration pipelines via an interactive visual DAG (Directed Acyclic Graph). The Node.js backend acts as a distributed saga coordinator that executes asynchronous task steps, handles automatic retries with exponential backoff, and triggers compensation transactions during downstream failure states.

Week-wise Plan

Week	Tasks
Week 1	• Architect multi-tenant database isolation strategies in MongoDB (discriminator keys vs. dynamic multi-database connection pools). • Develop a visual DAG workflow canvas in React with node validation, conditional branching, and loop detection. • Build Express middleware for tenant context routing, rate-limiting, and cryptographic API key management.
Week 2	• Construct a state-machine execution engine in Node.js to track workflow progression through distinct execution states. • Implement the Saga pattern to manage distributed rollbacks and compensation logic when external API webhooks fail. • Integrate MongoDB Change Streams to reactively detect event updates and trigger immediate workflow runs.
Mid-Project Review (End of Week 2)	☑ Multi-tenant data isolation and dynamic routing operational. ☑ Visual DAG canvas outputting validated execution graphs. ☑ Foundational Saga coordinator executing multi-step workflows with rollback capabilities.

Week 3	• Implement an isolated runtime sandbox (using isolated-vm or Worker Threads in Node.js) for executing user-defined transformation scripts safely • Establish bidirectional WebSocket streaming between the backend execution workers and the React client for live node execution tracing. • Implement dead-letter queues (DLQ) and configurable retry policies for transient third-party API errors.
Week 4	• Build an administrative dashboard in React to monitor tenant quotas, failure heatmaps, and worker latency. • Run stress testing against the orchestrator with 1,000+ concurrent distributed execution pipelines. • Containerize the system with Docker and establish auto-scaling worker groups using Kubernetes or AWS ECS.
Final Project Review (End of Week 4)	☑ Distributed compensation rollbacks and sandboxed scripts validated. ☑ Live execution tracing streaming seamlessly to frontend. ☑ Production-grade deployment and comprehensive failure-recovery documentation delivered.

Project 2: Distributed Cloud IDE & Sandboxed Code Execution Platform

Problem Statement:

Build a browser-based, zero-latency cloud integrated development environment (IDE). The frontend delivers a desktop-grade code editor with language intelligence, while the Express/Node.js backend orchestrates ephemeral, isolated Docker containers on demand, multiplexing raw terminal streams (PTY) and language server protocols (LSP) across low-latency WebSockets.

Week-wise Plan

Week	Tasks
Week 1	• Integrate the Monaco Editor into React with multi-file tab switching, syntax highlighting, and virtualized file tree explorer.

	• Design MongoDB schemas for project workspaces, file metadata trees, and execution environment templates. • Implement a container lifecycle manager in Node.js using the Docker Engine API to spawn ephemeral execution environments.
Week 2	• Bind virtual terminal sessions using xterm.js on the frontend and node-pty on the backend over persistent WebSockets. • Implement bidirectional standard I/O streaming (stdin, stdout, stderr) between the user's browser terminal and running containers. • Configure strict Linux cgroups, namespaces, and seccomp profiles to enforce container CPU, memory, and network limits.
Mid-Project Review (End of Week 2)	☒ Dynamic workspace creation and file tree persistence operational. ☒ Real-time interactive bash terminal functional in the browser via WebSockets. ☒ Container sandboxing rules validated against resource exhaustion exploits.
Week 3	• Implement Language Server Protocol (LSP) proxying over WebSockets to provide autocompletion and linting for multiple languages. • Build an internal micro-service to compile and run short scripts with sub-second execution latency, streaming logs back incrementally. • Add real-time multi-user pair-programming using CRDTs (Yjs) to keep file buffers synchronized between multiple active editors.
Week 4	• Implement client-side WebAssembly execution fallbacks in React for instant compilation of simple languages offline. • Perform security audits to ensure zero container escape or lateral network traversal from user code. • Package and deploy the multi-container platform with a reverse proxy (Nginx/Traefik) managing sticky WebSocket routing.
Final Project Review (End of Week 4)	☒ Full terminal multiplexing and remote LSP code completion functional. ☒ Secure multi-language sandbox with strict resource limits deployed.

	☒ End-to-end latency benchmarks and platform security report compiled.
--	--

Project 3: Zero-Knowledge Privacy-Preserving Document Verification Vault

Problem Statement:

Develop an enterprise-grade document compliance and sensitive identity verification platform. The application uses client-side Zero-Knowledge Proofs (ZKP via Circom/SnarkJS) and MongoDB Client-Side Field-Level Encryption (CSFLE) so organizations can cryptographically prove user credentials (e.g., age verification, accreditation, credit eligibility) without ever storing, seeing, or transmitting plaintext personally identifiable information (PII).

Week-wise Plan

Week	Tasks
Week 1	• Design ZK cryptographic circuits using Circom to generate proofs for specific numeric thresholds and identity assertions. • Configure MongoDB Client-Side Field-Level Encryption (CSFLE) utilizing an external Key Management Service (AWS KMS or HashiCorp Vault). • Build React client interfaces that handle local client-side key generation and secure document pre-processing.
Week 2	• Compile and integrate client-side ZK-SNARK proof generation within React using snarkjs and Web Workers. • Implement cryptographic proof verifiers in Node.js to validate assertions without decrypting the underlying data payload. • Construct an immutable Merkle tree ledger in MongoDB to detect any client-side document tampering.
Mid-Project Review (End of Week 2)	☒ ZK circuits compiled and verifying identity thresholds correctly. ☒ Client-Side Field-Level Encryption (CSFLE) enforced on the MongoDB database. ☒ Browser-based proof generation operational inside Web Workers without blocking UI threads.

Week 3	• Implement dynamic cryptographic revocation registries to invalidate expired credentials in real time. • Build a multi-party authorization workflow in Express requiring multiple verified approvals before updating encrypted records. • Develop a tamper-evident audit logging pipeline using SHA-256 cryptographic hash chains for every query transaction.
Week 4	• Construct an administrative portal in React with fine-grained Role-Based Access Control (RBAC) to inspect proof verification logs. • Optimize browser-based cryptographic proving times using WebAssembly multi-threading. • Conduct end-to-end security audits, verifying that zero raw PII leaks across network payloads or server logs.
Final Project Review (End of Week 4)	☒ Zero-Knowledge proof verification and automated revocation operational. ☒ Cryptographic audit trail and CSFLE-secured data layer verified. ☒ Production-ready deployment accompanied by a formal compliance and cryptography report.

Final Deliverables (All Projects)

- ☒ Distributed Workflow & Saga Orchestration Report
- ☒ Sandboxed Cloud IDE Architecture & Latency Report
- ☒ Zero-Knowledge Verification Vault Security Report

Tools & Technologies Used

- **Languages & Frameworks:** TypeScript, React, Node.js, Express.js
- **Databases & Encryption:** MongoDB (CSFLE, Change Streams), Redis, AWS KMS / HashiCorp Vault
- **Systems & Low-Level:** Docker API, node-pty, isolated-vm, Linux cgroups/seccomp, WebAssembly
- **Cryptography & Protocols:** SnarkJS, Circom, WebCrypto API, Merkle Trees, LSP, WebSockets
- **Editor & Collaboration:** Monaco Editor, xterm.js, Yjs (CRDTs), React Flow
- **DevOps & Deployment:** Docker Compose, Kubernetes, Traefik/Nginx, GitHub Actions

REQUIRED PROTOCOL FOR ALL ENGINEERING TEAMS

IMPORTANT INSTRUCTION FOR ALL TEAMS: Before commencing development, every team must create their respective GitHub repository. You are required to push commits daily for every week of the project lifecycle. This commit history will serve as the strict metric for your daily productivity tracking. Without consistent daily commits, your evaluation will not be processed by Zaalima Development Pvt.Ltd. Ensure professional commit messages, logical branch management, and adherence to standard collaborative workflows.

